



SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

SMART PHARMA SYSTEM

PROJECT REPORT FOR

**CSE23CL302 – COMPUTER AIDED SOFTWARE ENGINEERING
(CASE) TOOLS**

LABORATORY

Submitted by

BHARATRAJ-E0323054

YOKESHNANDHA-E0323041

In partial fulfillment for the award of the degree of
BACHELOR OF TECHNOLOGY

in

**COMPUTER SCIENCE AND ENGINEERING
(Artificial Intelligence and DATA ANALYTICS)**

**Sri Ramachandra Faculty of Engineering and Technology,
Sri Ramachandra Institute of Higher Education and
Research, Porur, Chennai -600116**

JUNE, 2026



SRI RAMACHANDRA
INSTITUTE OF HIGHER EDUCATION AND RESEARCH
(Category - I Deemed to be University) Porur, Chennai
SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

SMART PHARMA SYSTEM

PROJECT REPORT FOR

**CSE23CL302 – COMPUTER AIDED SOFTWARE ENGINEERING
(CASE) TOOLS**

LABORATORY

Submitted by

BHARATRAJ-E0323054

YOKESHNANDHA-E0323041

In partial fulfillment for the award of the degree of
BACHELOR OF TECHNOLOGY

in

**COMPUTER SCIENCE AND ENGINEERING
(Artificial Intelligence and DATA ANALYTICS)**

**Sri Ramachandra Faculty of Engineering and Technology,
Sri Ramachandra Institute of Higher Education and
Research, Porur, Chennai -600116**

JUNE, 2026



SRI RAMACHANDRA

INSTITUTE OF HIGHER EDUCATION AND RESEARCH

(Category - I Deemed to be University) Porur, Chennai

SRI RAMACHANDRA FACULTY OF ENGINEERING AND TECHNOLOGY

BONAFIDE CERTIFICATE

Certified that this project report titled **SMART PHARMA SYSTEM** is the bonafide record of work done by **BHARATRAJ-E0323054 AND YOKESHNANDHA-E0323041** who carried out the project work under my supervision.

Signature of the Supervisor

Yamini k
Assistant Professor,

Deoartmen of Cybersercurity and Internet of Things
Sri Ramachandra Faculty of Engineering and Technology,
SRIHER, Porur, Chennai-600 116.

INTERNAL EXAMINER

EXTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.		PAGE NO.
	ABSTRACT	5
1	PROBLEM DEFINITION	6
2	REQUIREMENTS ENGINEERING	9
3	DESIGN METHODOLOGY	15
4	CODING PRACTICES AND PLATFORMS	26
5	TESTING	32
6	DEPLOYMENT CHECK LIST	38
7	RESULTS	40
	APPENDIX	41

ABSTRACT

Smart Pharma System is a comprehensive web-based pharmaceutical production and management platform developed to automate and streamline the operational activities of pharmaceutical packaging industries. The system integrates inventory management, employee attendance tracking, salary calculation, task management, QR code and barcode scanning, audit logging, and reporting functionalities into a single centralized application.

The primary objective of the system is to eliminate manual record keeping, reduce operational errors, improve workforce productivity, and provide real-time monitoring of manufacturing activities. The platform enables administrators and managers to efficiently manage inventory items such as foils and cylinders, assign and monitor tasks, track employee attendance, calculate wages automatically, and generate analytical reports for decision-making.

The application is developed using React.js for the frontend, Node.js and Express.js for the backend, and MongoDB as the database. Security mechanisms such as JWT authentication, password encryption, role-based access control, and audit logging ensure data protection and system reliability.

The Smart Pharma System improves operational efficiency, transparency, accountability, and resource utilization within pharmaceutical manufacturing environments while providing a scalable foundation for future enhancements such as AI-based production forecasting and predictive analytics.

CHAPTER 1

PROBLEM DEFINITION

1.1 Introduction

The pharmaceutical packaging industry plays a critical role in ensuring the quality, safety, and traceability of medicines before they reach consumers. Effective management of inventory, workforce, production tasks, and operational records is essential for maintaining productivity and regulatory compliance. However, many pharmaceutical companies still rely on manual processes, spreadsheets, and disconnected software systems for managing day-to-day operations.

These traditional methods often lead to data inconsistencies, inventory mismatches, delayed task execution, attendance errors, and difficulties in generating accurate reports. Such challenges can negatively impact productivity, operational efficiency, and decision-making processes.

The Smart Pharma System has been developed to address these challenges by providing a centralized digital platform that integrates inventory management, employee management, attendance tracking, salary calculation, task management, QR/barcode-based tracking, and audit reporting functionalities.

1.2 Problem Statement

In pharmaceutical packaging industries, operational activities such as inventory control, employee attendance monitoring, task assignment, and production tracking are frequently managed through manual processes or independent software applications. This creates several issues including:

- Lack of centralized data management.
- Difficulty in tracking inventory movement.
- Inaccurate attendance and salary calculations.
- Poor visibility into employee performance.
- Delayed task monitoring and reporting.
- Increased chances of human errors.
- Limited accountability and audit tracking.
- Time-consuming report generation.

These limitations reduce operational efficiency and make it difficult for management to monitor organizational performance effectively.

1.3 Existing System

The existing system in many pharmaceutical organizations relies on manual documentation, spreadsheets, and separate software tools for managing operational activities.

Limitations of Existing System

- Manual inventory tracking.
- Paper-based attendance records.
- Separate systems for payroll and task management.

- High risk of data duplication and errors.
- Lack of real-time monitoring.
- No centralized reporting mechanism.
- Limited security and audit capabilities.
- Difficulty in maintaining historical records.

1.4 Proposed System

The Smart Pharma System is a centralized web application designed to automate and integrate various pharmaceutical operational processes into a single platform.

The system provides:

- User authentication and role-based access control.
- Inventory management for foils and cylinders.
- QR code and barcode generation and scanning.
- Employee attendance tracking.
- Automatic salary calculation.
- Task assignment and monitoring.
- Audit logging for compliance tracking.
- Dashboard analytics and reporting.
- Secure data management and storage.

By automating routine processes and providing real-time visibility into organizational activities, the system enhances productivity, accuracy, and decision-making.

1.5 Objectives of the System

The major objectives of the Smart Pharma System are:

- To automate pharmaceutical operational activities.
- To improve inventory tracking and management.
- To provide accurate attendance monitoring.
- To automate salary and wage calculations.
- To enable efficient task allocation and monitoring.
- To maintain audit logs for accountability.
- To generate real-time analytical reports.
- To improve productivity and operational efficiency.
- To reduce manual errors and paperwork.
- To ensure data security and reliability.

1.6 Scope of the Project

The scope of the Smart Pharma System includes:

- Employee Management

- Attendance Management
- Inventory Management
- Task Management
- Salary Processing
- QR/Barcode Tracking
- Audit Logging
- Reporting and Analytics
- User Authentication and Authorization

The system is designed for pharmaceutical packaging industries and can be expanded in the future to support advanced production monitoring, AI-based forecasting, and enterprise resource planning functionalities.

1.7 Benefits of the Proposed System

The proposed system offers the following benefits:

- Centralized operational management.
- Faster and more accurate data processing.
- Improved inventory visibility.
- Reduced administrative workload.
- Enhanced employee accountability.
- Secure data storage and access control.
- Automated report generation.
- Better decision-making through analytics.
- Scalability for future organizational growth.

1.8 Conclusion

The Smart Pharma System addresses the operational challenges faced by pharmaceutical packaging industries by integrating multiple management functions into a unified platform. Through automation, centralized data management, and real-time monitoring capabilities, the system improves efficiency, transparency, and organizational productivity while reducing operational costs and human errors.

CHAPTER 2

REQUIREMENTS ENGINEERING

2.1 Requirements Engineering Life Cycle

Requirements Engineering is a systematic process used to identify, document, analyze, validate, and manage software requirements throughout the project lifecycle. It ensures that the developed system satisfies user expectations and business objectives.

The Requirements Engineering Life Cycle for the Smart Pharma System consists of the following phases:

Requirement Elicitation

Information is gathered from stakeholders such as administrators, managers, supervisors, and employees through interviews, discussions, observations, and document analysis.

Requirement Analysis

The collected requirements are analyzed to identify conflicts, dependencies, priorities, and feasibility.

Requirement Specification

All requirements are documented in a structured format known as the Software Requirements Specification (SRS).

Requirement Validation

Requirements are reviewed and validated with stakeholders to ensure correctness, completeness, consistency, and feasibility.

Requirement Management

Requirements are continuously monitored and updated throughout the software development lifecycle.

2.2 Categorizing Different Types of Requirements

2.2.1 Functional Requirements

Functional requirements describe the specific functionalities that the system must perform.

User Management

- User Registration
- User Login

- Password Encryption
- Role-Based Access Control

Inventory Management

- Add Inventory Items
- Update Inventory Records
- Delete Inventory Records
- Generate QR Codes
- Generate Barcodes

Attendance Management

- Employee Check-In
- Employee Check-Out
- Attendance History Tracking

Salary Management

- Salary Rate Configuration
- Automatic Salary Calculation
- Salary Report Generation

Task Management

- Create Tasks
- Assign Tasks
- Update Task Status
- Complete Tasks

Reporting Module

- Attendance Reports
- Salary Reports
- Inventory Reports
- Audit Reports

Audit Management

- Log User Activities
- Monitor System Events
- Track Changes

2.2.2 Non-Functional Requirements

Non-functional requirements define the quality attributes of the system.

Performance Requirements

- API response time should be less than 3 seconds.
- Dashboard data should load efficiently.

Security Requirements

- JWT-based Authentication.
- Password Encryption using bcrypt.
- Role-Based Access Control.

Reliability Requirements

- Data backup mechanisms.
- Error handling and recovery.

Scalability Requirements

- Support increasing numbers of users and inventory records.

Availability Requirements

- System availability of at least 99%.

Maintainability Requirements

- Modular code architecture.
- Proper documentation and logging.

Usability Requirements

- User-friendly interface.
- Responsive design for different devices.

2.2.3 Domain Requirements

Domain requirements are specific to the pharmaceutical packaging industry.

- Pharmaceutical inventory tracking.
- Foil management.
- Cylinder management.
- QR code validation.
- Barcode-based inventory identification.

- Production task monitoring.
- Employee attendance tracking.
- Salary computation based on working hours.
- Audit trail maintenance.
- Compliance reporting.

2.3 Requirements Traceability Matrix

The Requirements Traceability Matrix (RTM) helps track each requirement throughout development and testing.

Tab 2.1 Requirements Traceability Matrix

Requirement ID	Requirement Description	Design Module	Test Case	Status
FR-001	User Registration	Authentication	TC-01	Completed
FR-002	User Login	Authentication	TC-02	Completed
FR-003	Inventory Management	Inventory	TC-03	Completed
FR-004	QR Code Generation	Inventory	TC-04	Completed
FR-005	Attendance Tracking	Attendance	TC-05	Completed
FR-006	Salary Calculation	Salary	TC-06	Completed

FR-007	Task Assignment	Task Management	TC-07	Completed
FR-008	Report Generation	Reporting	TC-08	Completed
NFR-001	Secure Authentication	Security	TC-09	Completed
NFR-002	System Performance	Performance	TC-10	Completed

2.4 Hardware Requirements

The following hardware specifications are required for developing and deploying the Smart Pharma System.

Hardware Component	Minimum Requirement	Recommended Requirement
Processor	Intel Core i3	Intel Core i5/i7
RAM	8 GB	16 GB
Storage	256 GB HDD	512 GB SSD
Display	1366 × 768	1920 × 1080
Internet	10 Mbps	50 Mbps
QR Scanner	Optional	Recommended
Barcode Scanner	Optional	Recommended

2.5 Software Requirements

The software tools used for developing the Smart Pharma System are listed below.

Tab 2.3 Software Requirements

Category	Software / Tool	Version	Purpose
Operating System	Windows	10/11	Development Platform
Frontend	React.js	18.x	User Interface Development
Backend	Node.js	18.x	Server Runtime
Framework	Express.js	4.x	API Development
Database	MongoDB	6.x	Data Storage
Authentication	JWT	Latest	User Authentication
Encryption	bcrypt	Latest	Password Security
Testing Tool	Postman	Latest	API Testing
IDE	Visual Studio Code	Latest	Development Environment
Version Control	Git & GitHub	Latest	Source Code Management
Logging	Winston	Latest	Activity Logging
Package Manager	npm	Latest	Dependency Management

CHAPTER 3

DESIGN ENGINEERING

3.1 Design Diagrams (UML Diagrams)

3.1.1 Use Case Diagram

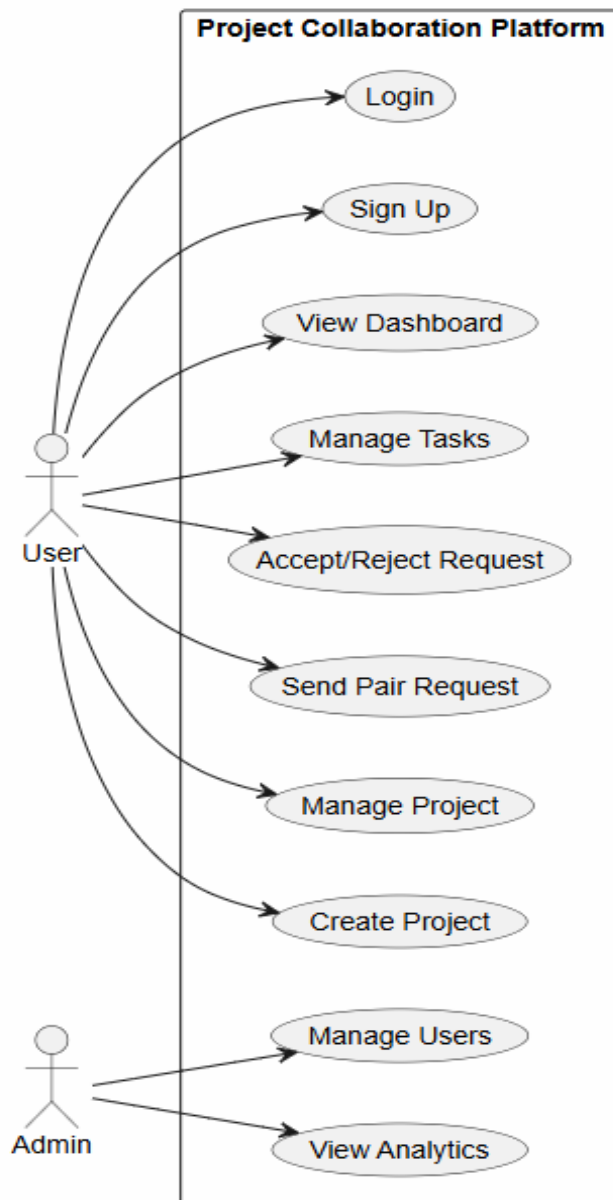


Fig 3.1 Use Case Diagram

3.1.2 Class Diagram

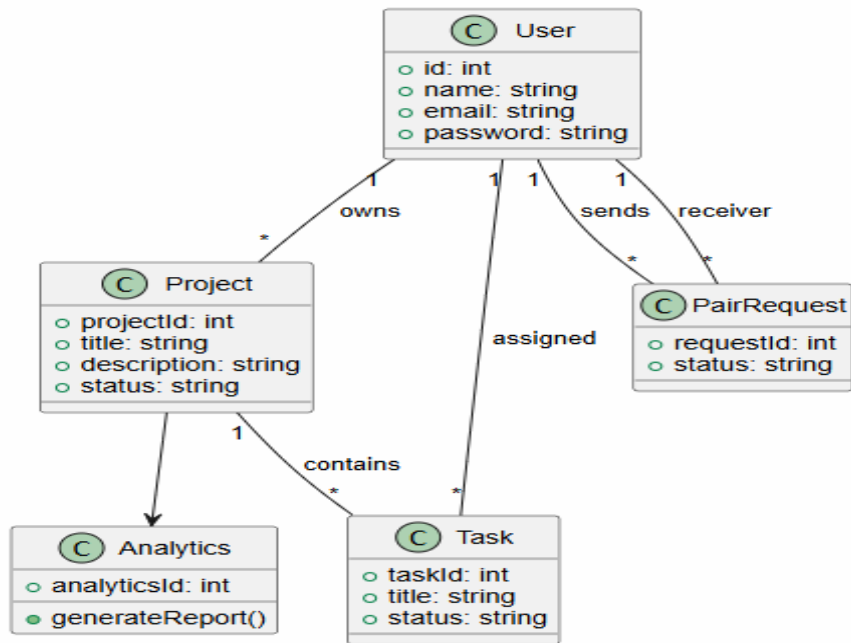


Fig 3.2 Class Diagram

3.1.3 Sequence Diagram

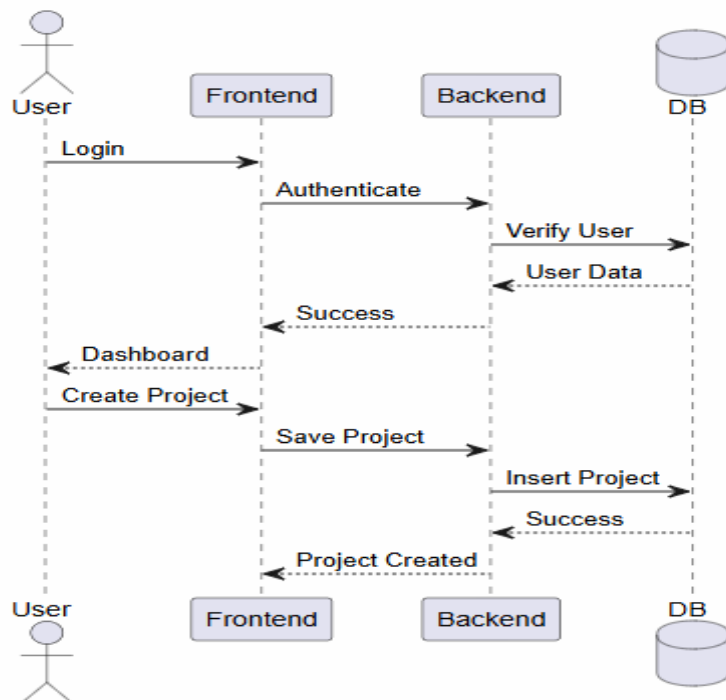


Fig 3.3 Sequence Diagram

3.1.4 Activity Diagram

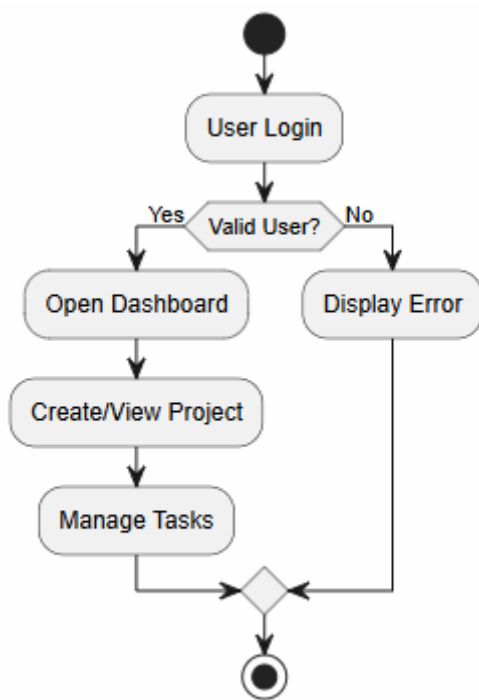


Fig 3.4 Activity Diagram

3.1.5 State Diagram — PairRequest

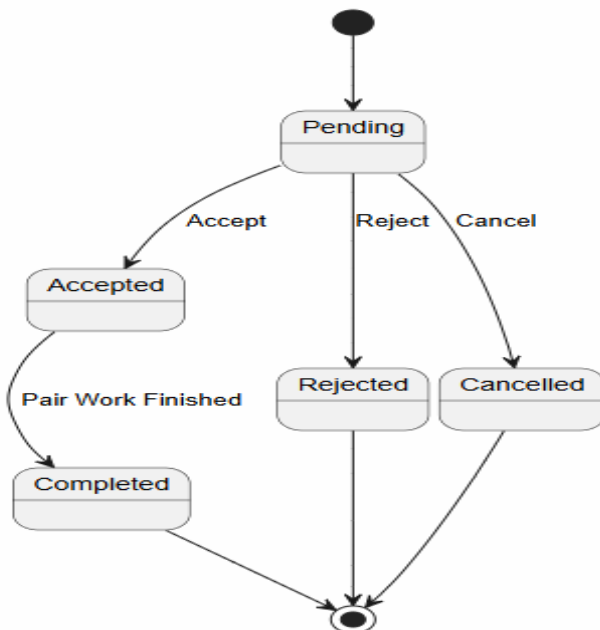


Fig 3.5 State Diagram — PairRequest

3.1.6 State Diagram — Project

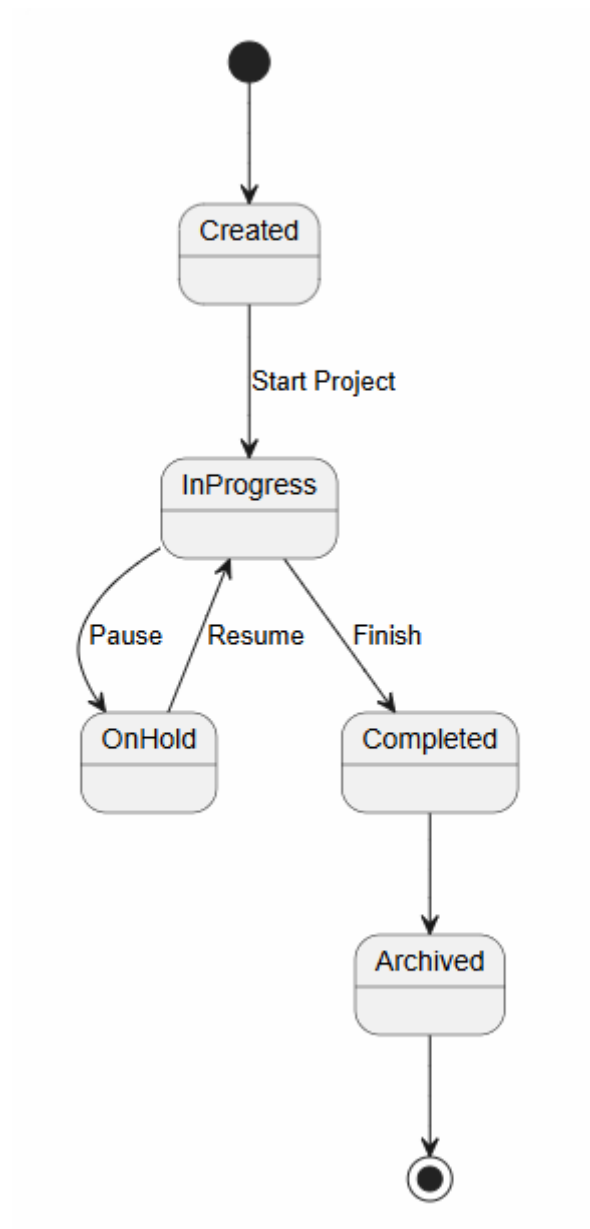


Fig 3.6 State Diagram — Project

3.1.7 ER Diagram

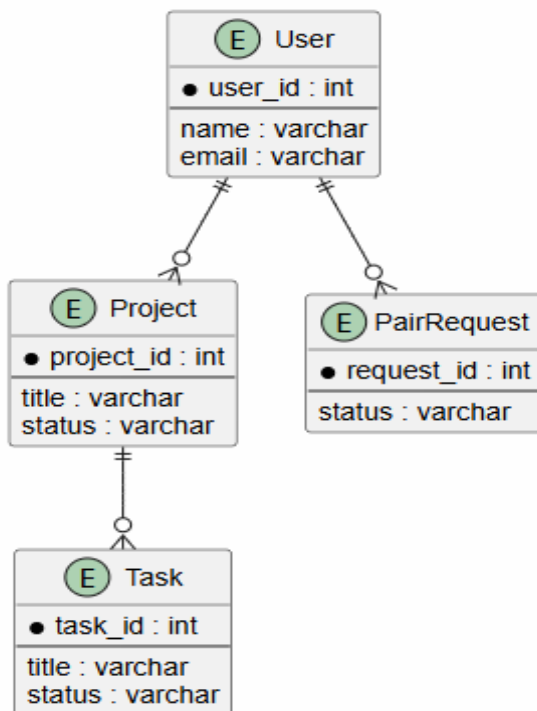


Fig 3.7 ER Diagram

3.1.8 Component / Dependency Diagram

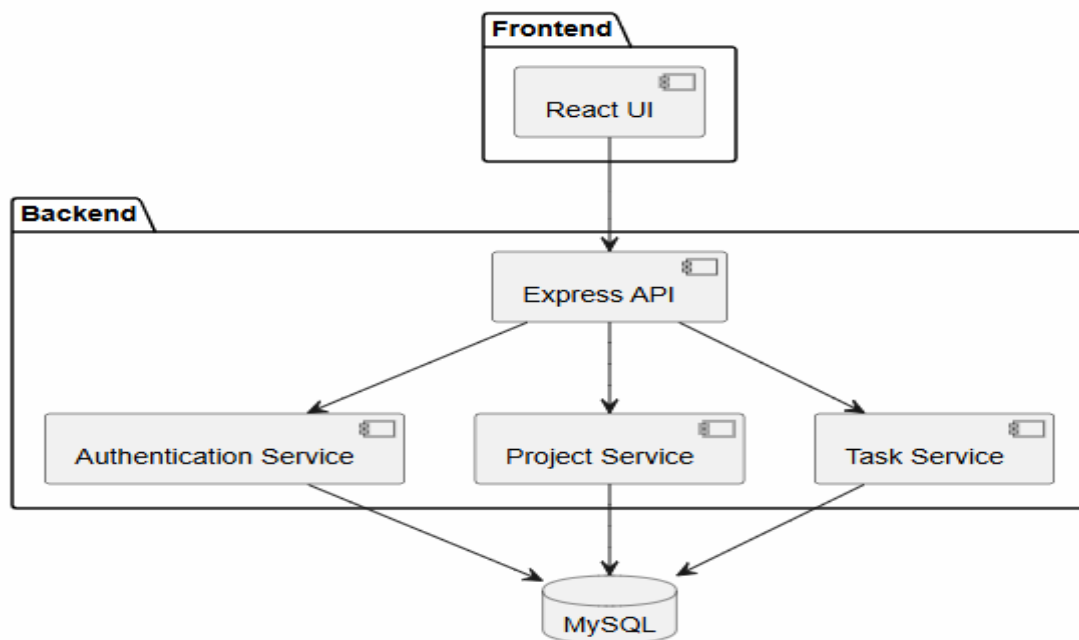


Fig 3.8 Component / Dependency Diagram

3.2 Cohesion and Coupling

Cohesion and Coupling are important software design principles used to improve system maintainability, scalability, and reliability. The Smart Pharma System is designed with high cohesion and low coupling to ensure better software quality.

3.2.1 Cohesion (High Cohesion)

Cohesion refers to the degree to which the elements within a module belong together. A highly cohesive module performs a single well-defined task and contains closely related functionalities. The Smart Pharma System follows the principle of High Cohesion by dividing the application into specialized modules.

Authentication Module

Responsibilities:

- User Registration
- User Login
- Password Management
- JWT Authentication

Inventory Module

Responsibilities:

- Foil Management
- Cylinder Management
- QR Code Generation
- Barcode Generation

Attendance Module

Responsibilities:

- Check-In
- Check-Out
- Attendance Tracking
- Working Hours Calculation

Salary Module

Responsibilities:

- Salary Calculation
- Earnings Tracking
- Payroll Reporting

Task Management Module

Responsibilities:

- Task Creation
- Task Assignment
- Task Status Updates

- Task Completion Tracking

Reporting Module

Responsibilities:

- Attendance Reports
- Inventory Reports
- Salary Reports
- Audit Reports

Benefits of High Cohesion

- Easy maintenance
- Improved readability
- Better reusability
- Reduced complexity
- Easier testing and debugging

3.2.2 Coupling (Low Coupling)

Coupling refers to the degree of dependency between software modules. Low Coupling means modules interact with each other through well-defined interfaces while remaining independent. The Smart Pharma System follows Low Coupling architecture through REST APIs and service-based communication.

Examples of Low Coupling

Authentication and Inventory

The Inventory Module only verifies authentication tokens and does not directly access authentication logic.

Attendance and Salary

Salary calculations use attendance data through service calls without tightly connecting both modules.

Reporting Module

The Reporting Module collects data from multiple modules without modifying their internal functionality.

Benefits of Low Coupling

- Easy module replacement
- Better scalability
- Independent development
- Simplified maintenance
- Reduced risk of system-wide failures

3.3 Design Pattern

The Smart Pharma System adopts the Model-View-Controller (MVC) Design Pattern.

Model Layer

The Model layer manages data and business entities.

Examples:

- User Model
- Attendance Model
- Task Model
- Inventory Model
- Audit Log Model

View Layer

The View layer provides the user interface using React.js.

Examples:

- Login Page
- Dashboard
- Inventory Page
- Attendance Page
- Reports Page

Controller Layer

The Controller layer processes user requests and communicates with the database.

Examples:

- Authentication Controller
- Inventory Controller
- Attendance Controller
- Salary Controller
- Report Controller

Advantages of MVC Pattern

- Separation of concerns
- Better code organization
- Easier maintenance
- Improved scalability
- Enhanced testability

Additional patterns used include:

- Service Layer Pattern
- Repository Pattern
- Middleware Pattern
- Dependency Injection Concepts

3.4 Usage of Appropriate Architectural Styles

The Smart Pharma System follows a Three-Tier Client-Server Architecture.

Presentation Layer

The Presentation Layer is responsible for user interaction and interface rendering.

Technologies:

- React.js
- HTML5
- CSS3
- JavaScript

Functions:

- User Interface
- Dashboard Visualization
- Form Handling
- Report Display

Business Logic Layer

The Business Logic Layer processes application rules and workflows.

Technologies:

- Node.js
- Express.js

Functions:

- Authentication
- Inventory Processing
- Attendance Processing
- Salary Calculation
- Report Generation

Data Layer

The Data Layer stores and manages application data.

Technologies:

- MongoDB
- Mongoose ODM

Functions:

- Data Storage
- Data Retrieval
- Data Validation
- Database Operations

Architectural Benefits

- Scalability
- Security

- Flexibility
- Maintainability
- High Performance
- Easy Deployment

3.5 Interface Design

The Smart Pharma System provides a simple, responsive, and user-friendly interface that enables users to perform daily operations efficiently.

Login Interface

Features:

- Username Field
- Password Field
- Login Button
- Forgot Password Option

Purpose:

Provides secure authentication and access control.

Dashboard Interface

Features:

- Employee Statistics
- Inventory Summary
- Attendance Summary
- Task Overview
- Report Widgets

Purpose:

Provides a centralized overview of system activities.

Inventory Management Interface

Features:

- Add Inventory
- Edit Inventory
- Delete Inventory
- QR Code Generation
- Barcode Generation

Purpose:

Manages pharmaceutical inventory records efficiently.

Attendance Management Interface

Features:

- Employee Check-In

- Employee Check-Out
- Attendance History
- Working Hour Tracking

Purpose:

Monitors employee attendance and productivity.

Salary Management Interface

Features:

- Salary Configuration
- Salary Calculation
- Payroll Reports

Purpose:

Automates payroll processing.

Task Management Interface

Features:

- Create Task
- Assign Task
- Update Status
- View Progress

Purpose:

Improves workflow management and employee coordination.

Reporting Interface

Features:

- Attendance Reports
- Salary Reports
- Inventory Reports
- Audit Logs

Purpose:

Supports management decision-making through analytical reports.

Interface Design Principles

- Simplicity
- Consistency
- Responsiveness
- Accessibility
- Security
- User Friendliness

The interface design ensures that users can perform their tasks efficiently while maintaining a professional and organized user experience.

CHAPTER 4

CODING PRACTICES

Coding practices play a significant role in developing reliable, maintainable, scalable, and secure software systems. The Smart Pharma System follows industry-standard coding practices, including static code analysis, dynamic code analysis, logging mechanisms, runtime monitoring, validation techniques, and security controls to ensure software quality.

4.1 Static Code Analysis

Static Code Analysis is the process of examining source code without executing the application. It helps identify coding errors, security vulnerabilities, code smells, and maintainability issues during development.

Benefits include:

- Early defect detection
- Improved code quality
- Consistent coding standards
- Enhanced maintainability
- Reduced debugging effort

The Smart Pharma System utilizes various static analysis tools to maintain high-quality code.

4.1.1 ESLint (Frontend)

ESLint is a static code analysis tool used to identify problematic patterns and coding errors in JavaScript and React applications.

Purpose

The frontend of the Smart Pharma System is developed using React.js. ESLint is used to enforce coding standards and improve code consistency across the project.

Features

- Detects syntax errors
- Enforces coding conventions
- Prevents unused variables
- Identifies potential bugs
- Maintains code readability

Example Checks

- Missing semicolons
- Undefined variables
- Duplicate declarations
- Unused imports

- Improper React Hook usage

Benefits

- Improved code quality
- Better maintainability
- Reduced runtime errors
- Consistent coding practices

4.1.2 express-validator (Backend)

express-validator is a middleware library used for validating and sanitizing user input in Express.js applications.

Purpose

The Smart Pharma System uses express-validator to ensure that incoming API requests contain valid and secure data before processing.

Validation Examples

- Email validation
- Password validation
- Required field validation
- Numeric input validation
- Data sanitization

Example

User Registration API validates:

- Username
- Email Address
- Password Length
- User Role

Benefits

- Prevents invalid data entry
- Enhances security
- Reduces server-side errors
- Protects against malicious inputs

4.2 Dynamic Code Analysis

Dynamic Code Analysis involves monitoring the application while it is running. It helps detect runtime errors, performance issues, security vulnerabilities, and system behavior under different conditions.

The Smart Pharma System uses several runtime monitoring mechanisms to improve reliability and performance.

4.2.1 Winston Logger

Winston is a powerful logging library used for recording application activities and runtime events.

Purpose

The Smart Pharma System uses Winston Logger to track:

- User activities
- API requests
- System errors
- Security events
- Audit logs

Logged Information

- Login attempts
- Inventory updates
- Attendance modifications
- Task assignments
- Error messages

Benefits

- Easy debugging
- Error tracking
- Performance monitoring
- Security auditing

4.2.2 Health Check Endpoint

A Health Check Endpoint is used to verify whether the application and its services are functioning correctly.

Endpoint Example

/api/health

Checks Performed

- Server Status
- Database Connectivity
- API Availability
- Memory Usage

- System Uptime

Sample Response

Status: Healthy

Database: Connected

Server: Running

Benefits

- Continuous monitoring
- Quick fault detection
- Deployment verification
- System reliability assurance

4.2.3 Postman API Testing

Postman is used to test and validate REST APIs during development and deployment.

Purpose

The Smart Pharma System uses Postman to verify:

- API functionality
- Response validation
- Authentication workflows
- Error handling

APIs Tested

- Login API
- Registration API
- Inventory API
- Attendance API
- Salary API
- Reporting API

Benefits

- Automated testing
- Faster debugging
- API verification
- Reduced integration errors

4.2.4 Sequelize Query Logging

Sequelize Query Logging records database queries executed by the application during runtime.

Purpose

Query logging helps developers monitor database interactions and identify performance bottlenecks.

Information Logged

- SELECT queries
- INSERT queries
- UPDATE queries
- DELETE queries
- Query execution time

Benefits

- Database performance optimization
- Slow query detection
- Easier debugging
- Improved transparency

Note

If MongoDB and Mongoose are used instead of Sequelize, similar query monitoring can be implemented using Mongoose debugging features.

4.2.5 Rate Limiting (Runtime Protection)

Rate Limiting restricts the number of requests that a user can send to the server within a specified time period.

Purpose

Protects the Smart Pharma System from:

- Brute force attacks
- DDoS attacks
- API abuse
- Excessive resource consumption

Example Configuration

- Maximum Requests: 100
- Time Window: 15 Minutes

Benefits

- Enhanced security
- Improved server stability
- Protection against abuse
- Better resource utilization

4.2.6 Sequelize Transaction Monitoring

Transaction Monitoring ensures database operations execute reliably and maintain data consistency.

Purpose

The Smart Pharma System monitors critical transactions such as:

- Inventory Updates
- Salary Calculations
- Attendance Processing
- Task Assignments

Monitoring Features

- Transaction Start
- Transaction Commit
- Transaction Rollback
- Error Detection

Benefits

- Data consistency
- Error recovery
- Reliable operations
- Improved database integrity

Note

For MongoDB-based implementations, transaction monitoring can be performed using MongoDB Sessions and Mongoose Transactions.

.

CHAPTER 5

TESTING

Testing is an essential phase of software development that ensures the Smart Pharma System functions correctly, meets user requirements, and operates reliably under different conditions.

5.1 Debugging Techniques

The following debugging techniques were used during development:

Console Logging

Used to monitor variable values and execution flow.

Browser Developer Tools

Used for frontend debugging, network monitoring, and performance analysis.

Postman Testing

Used to verify API responses and backend functionality.

Error Logging

Winston Logger was used to record application errors and runtime events.

Database Query Inspection

Database operations were monitored to identify incorrect queries and data inconsistencies.

Breakpoint Debugging

Visual Studio Code debugger was used to inspect code execution step-by-step.

5.2 Unit Testing

Unit testing verifies individual functions and modules independently.

Tab 5.2 Unit Testing

Module	Function Tested	Input	Expected Output	Result
Authentication	User Login	Valid Username & Password	Successful Login	Pass
Authentication	User Login	Invalid Credentials	Authentication Error	Pass
Authentication	User Registration	Valid User Data	User Created	Pass
Inventory	Add Foil	Valid Foil Details	Foil Added Successfully	Pass
Inventory	Update Foil	Updated Data	Foil Updated	Pass
Inventory	Delete Foil	Foil ID	Foil Deleted	Pass
Attendance	Check-In	Employee ID	Attendance Recorded	Pass
Attendance	Check-Out	Employee ID	Check-Out Recorded	Pass
Salary	Salary Calculation	Attendance Hours	Correct Salary Generated	Pass
Task Management	Create Task	Task Details	Task Created	Pass
Task Management	Assign Task	Employee ID	Task Assigned	Pass
Reports	Generate Report	Date Range	Report Generated	Pass
QR Module	QR Generation	Inventory Data	QR Code Generated	Pass
Barcode Module	Barcode Generation	Cylinder Data	Barcode Generated	Pass
Audit Log	Activity Logging	User Action	Log Created	Pass

5.3 Integration Testing

Integration testing verifies the interaction between multiple modules.

Tab 5.3

Modules Integrated	Scenario Tested	Expected Outcome	Result
Authentication + Dashboard	Login and Dashboard Access	Dashboard Loaded Successfully	Pass
Authentication + Inventory	Authorized Inventory Access	Inventory Displayed	Pass
Inventory + QR Module	Generate QR After Inventory Entry	QR Created Successfully	Pass
Inventory + Barcode Module	Generate Barcode for Cylinder	Barcode Generated	Pass
Attendance + Salary	Calculate Salary from Attendance	Correct Salary Generated	Pass
Task + Employee Module	Assign Task to Employee	Task Assigned Successfully	Pass
Reports + Database	Generate Attendance Report	Report Generated	Pass
Audit Log + Authentication	Record Login Activity	Log Created	Pass

Dashboard + Reports	Display Report Statistics	Analytics Displayed	Pass
Inventory + Audit Log	Inventory Update Tracking	Audit Record Generated	Pass

5.4 System Testing

System testing evaluates the complete Smart Pharma System as a whole.

Tab 5.4 System Testing

Test Scenario	Expected Result	Actual Result	Status
User Registration	New User Created Successfully	User Created Successfully	Pass
User Login	Access Granted	Access Granted	Pass
Inventory Creation	Inventory Stored in Database	Inventory Stored Successfully	Pass
QR Code Generation	QR Code Generated	QR Code Generated	Pass
Barcode Generation	Barcode Generated	Barcode Generated	Pass
Attendance Check-In	Attendance Recorded	Attendance Recorded	Pass
Attendance Check-Out	Working Hours Calculated	Working Hours Calculated	Pass
Salary Calculation	Salary Generated Correctly	Salary Generated Correctly	Pass

Task Assignment	Task Assigned Successfully	Task Assigned Successfully	Pass
Report Generation	Reports Displayed	Reports Displayed	Pass
Audit Logging	Activity Logged	Activity Logged	Pass

5.5 Bug Report

The following bugs were identified and resolved during development.

Tab 5.4 Bug Report

Bug ID	Module	Bug Description	Severity	Steps to Reproduce	Resolution	Status
BUG-001	Authentication	Invalid JWT Token Error	High	Login with expired token	Token Refresh Logic Added	Closed
BUG-002	Inventory	Duplicate Inventory Entry	Medium	Add same inventory twice	Duplicate Validation Added	Closed
BUG-003	Attendance	Check-Out Time Not Recorded	High	Employee Check-Out	Fixed Database Update Logic	Closed
BUG-004	Salary	Incorrect Salary Calculation	High	Calculate Salary	Corrected Formula Logic	Closed
BUG-005	Dashboard	Statistics Not Refreshing	Medium	Update Inventory	Added Auto Refresh API	Closed
BUG-006	Reports	Report Export Failure	Medium	Export PDF Report	Fixed Export Service	Closed
BUG-007	QR Module	QR Scan Validation Error	Medium	Scan Invalid QR	Improved Validation Logic	Closed
BUG-008	Barcode Module	Barcode Not Rendering	Low	Generate Barcode	Updated Barcode Library	Closed
BUG-009	Audit Log	Missing User Activity Logs	Medium	Perform User Action	Fixed Logging Middleware	Closed
BUG-010	API Gateway	Slow API Response	Medium	Multiple Requests	Query Optimization Applied	Closed

CHAPTER 6

DEPLOYMENT CHECKLIST

Tab 6.1 Deployment Checklist

S.No	Deployment Activity	Status
1	Install Node.js Environment	Completed
2	Install MongoDB Database	Completed
3	Configure Environment Variables	Completed
4	Install Backend Dependencies	Completed
5	Install Frontend Dependencies	Completed
6	Configure JWT Secrets	Completed
7	Configure Database Connection	Completed
8	Build React Application	Completed
9	Execute Unit Testing	Completed

S.No	Deployment Activity	Status
10	Execute Integration Testing	Completed
11	Execute System Testing	Completed
12	Verify API Endpoints	Completed
13	Enable Logging Services	Completed
14	Configure Security Settings	Completed
15	Deploy Backend Server	Completed
16	Deploy Frontend Application	Completed
17	Verify Production Environment	Completed
18	Monitor Application Health	Completed

CHAPTER 7

RESULT

The Smart Pharma System was successfully developed and tested to automate pharmaceutical inventory management, employee attendance tracking, salary calculation, task management, QR/barcode processing, audit logging, and reporting activities.

The system achieved the following outcomes:

- Successful user authentication and authorization.
- Efficient inventory management of foils and cylinders.
- Accurate attendance tracking and salary computation.
- Automated QR code and barcode generation.
- Effective task assignment and monitoring.
- Comprehensive audit logging and reporting.
- Real-time dashboard analytics.
- Secure and scalable architecture.

The testing results confirmed that all major functionalities performed as expected with minimal errors. The system improved operational efficiency, reduced manual work, enhanced data accuracy, and provided better decision-making support through centralized management and analytical reporting.

Therefore, the Smart Pharma System successfully meets the objectives defined during the requirements engineering phase and provides a reliable solution for pharmaceutical production and workforce management.

APPENDIX

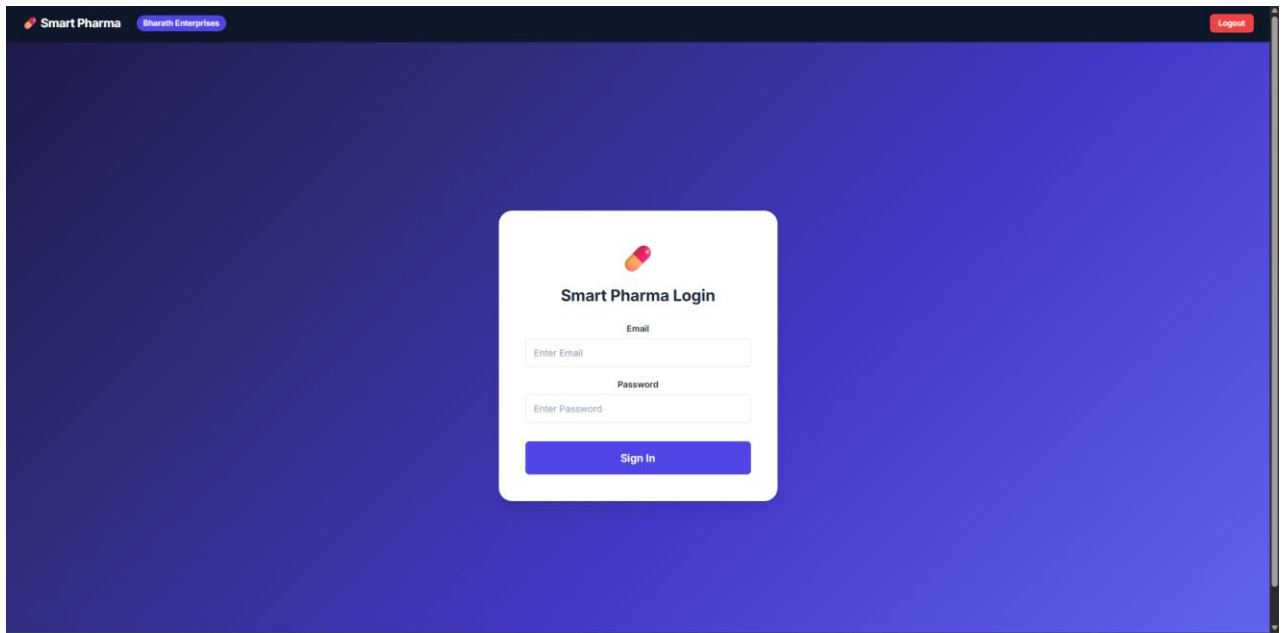


Fig 7.1 signup page

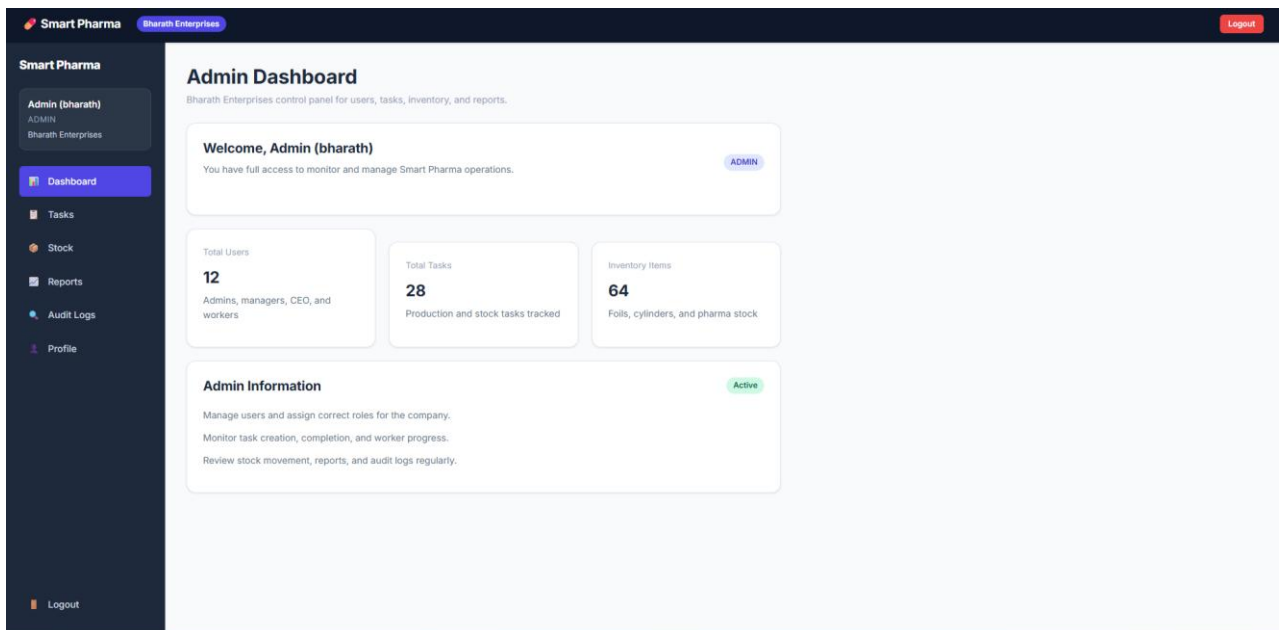


Fig 7.2 Admin Dashboard

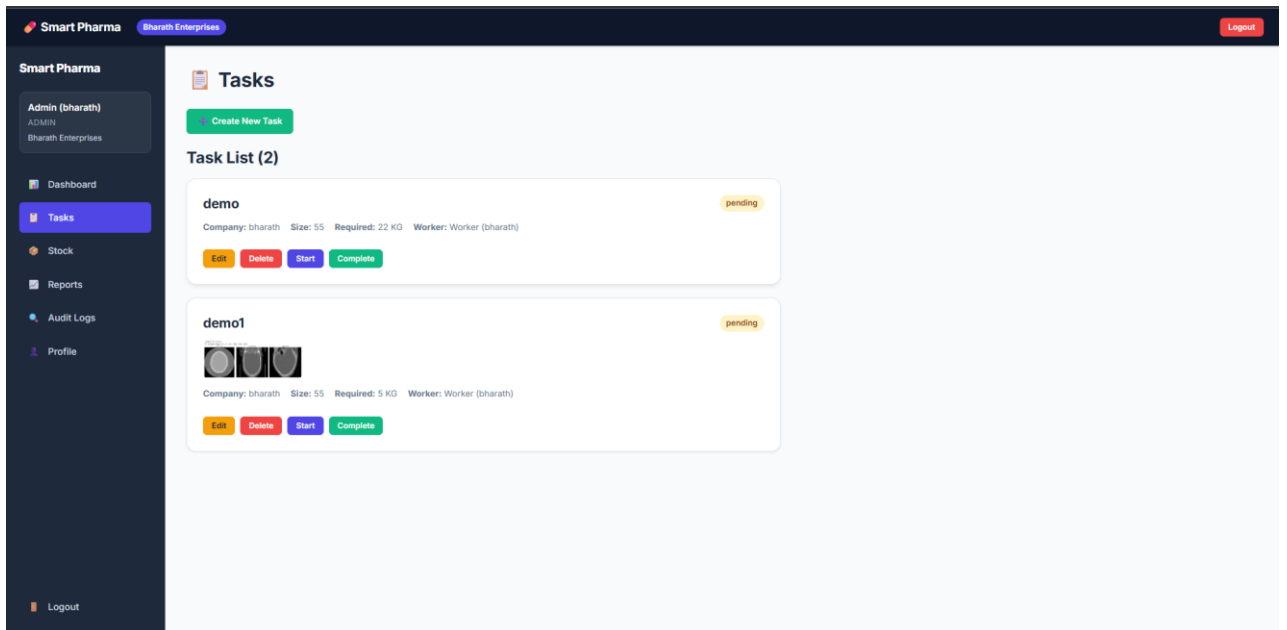


Fig 7.3 task manager

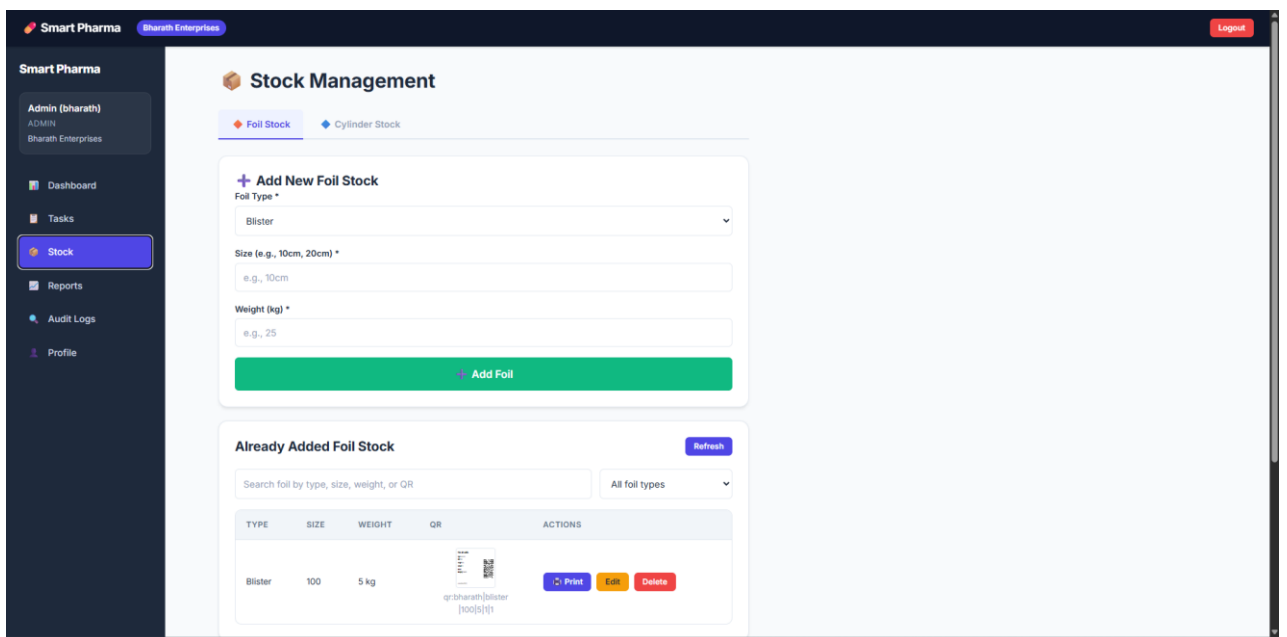


Fig 7.4 Stock management

FOIL LABEL

Type:

BLISTER

Size:

100

Weight:

5 KG

qr:bharath|blister|100|5|1|1

FOIL QR LABEL

Type: BLISTER

Size: 100

Weight: 5KG

Version: V1

Serial: 1

Company: bharath



qr:bharath|blister|100|5|1|1

Date: 6/5/2026 11:56:38 PM

 Print Label

 Close

Fig 7.5 bar code

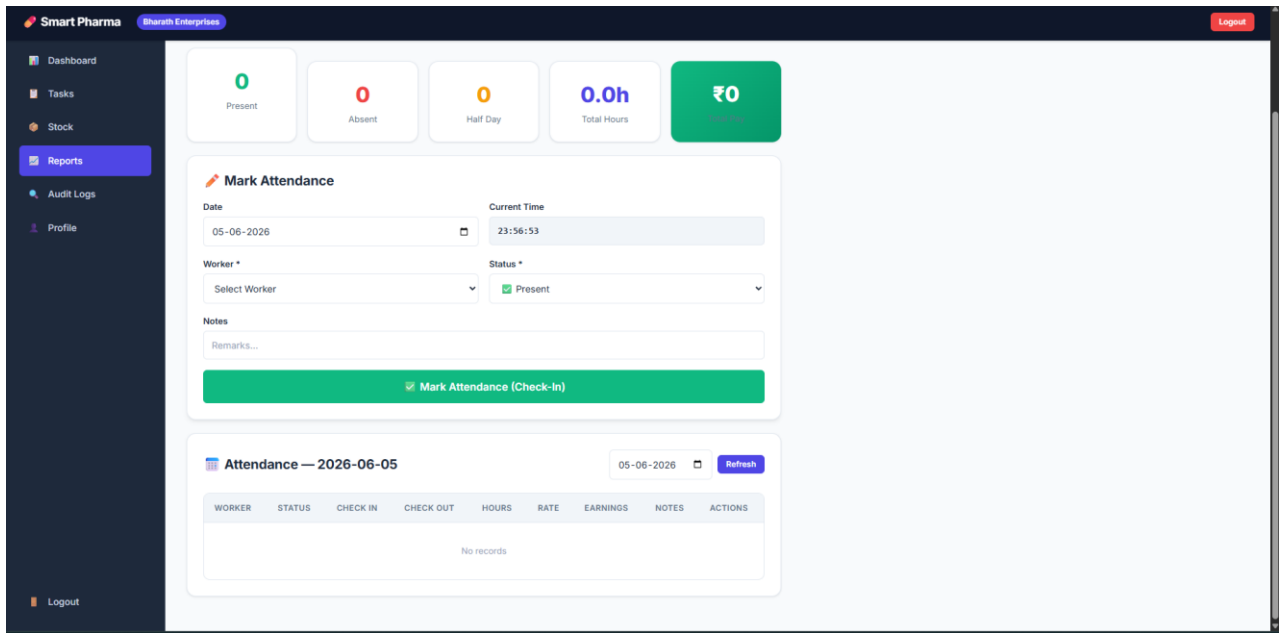


Fig 7.6 Attendance report

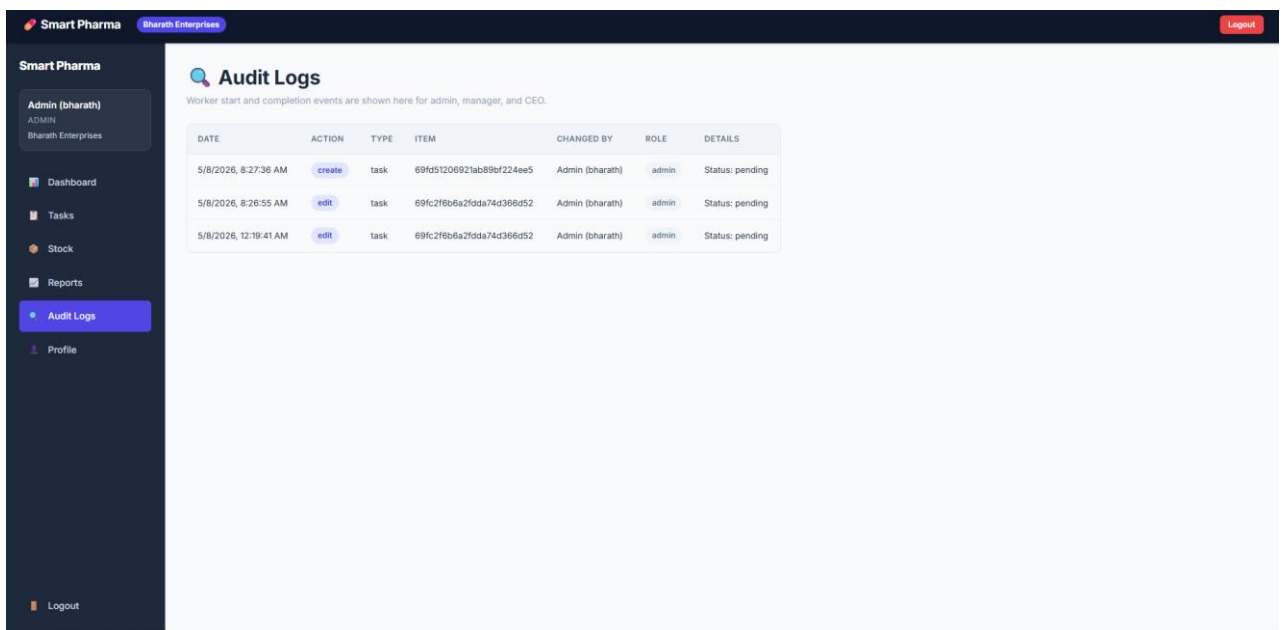


Fig 7.7 Audit logs

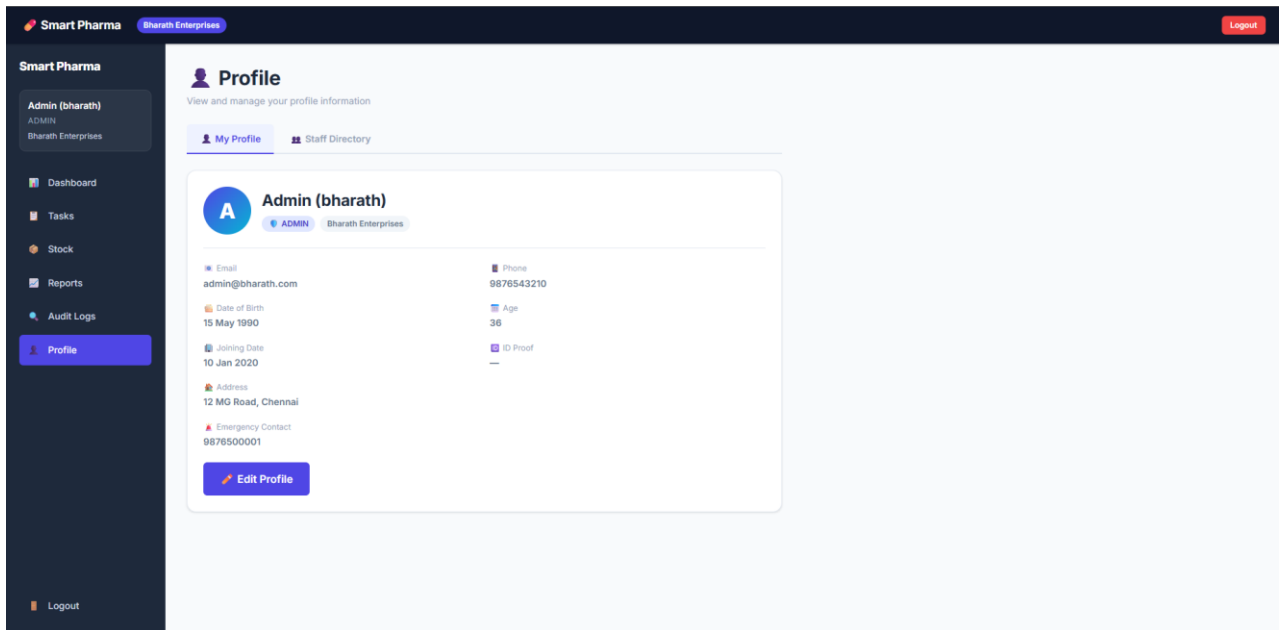


Fig 7.8 Profile settings